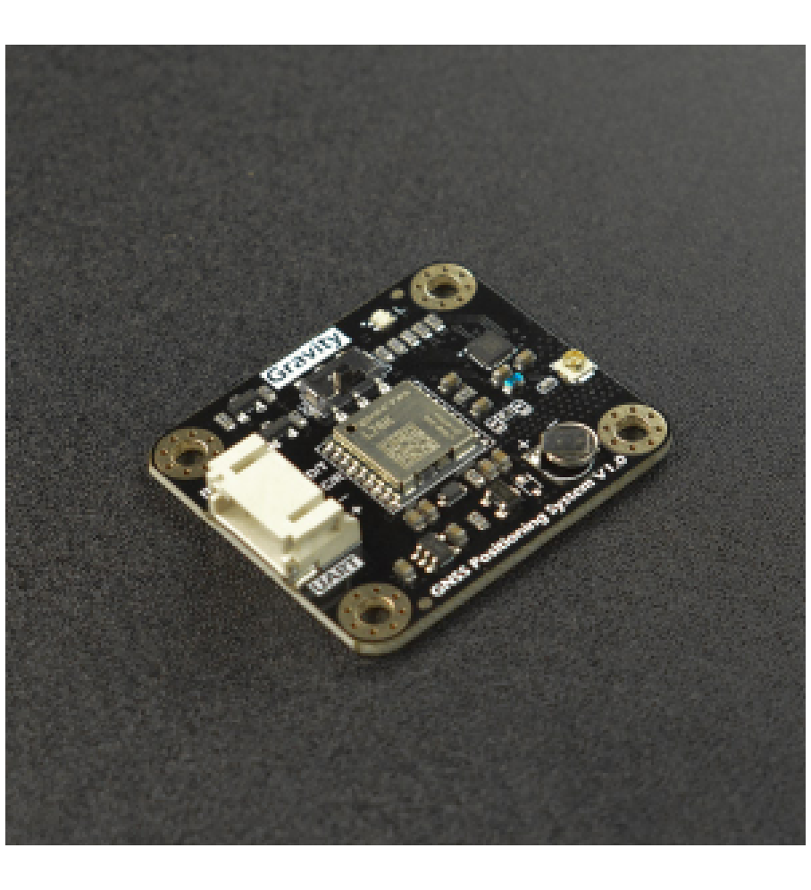
search

Controller
DFR0182 Wirless GamePad V2.0
DFR0100 DFRduino Beginner Kit For Arduino V3
DFR0267 Bluno
DFR0282 Beetle
DFR0283 Dreamer Maple V1.0
DFR0296 Bluno Nano
DFR0302 MiniQ 2WD Plus
DFR0304 BLE Wireless Gamepad V2
DFR0305 RoMeo BLE
DFR0351 Romeo BLE mini V2.0
DFR0306 Bluno Mega 1280
DFR0321 Wido-WIFI IoT Node
DFR0323 Bluno Mega 2560
DFR0329 Bluno M3
DFR0339 Bluno Beetle
DFR0343 UHex Low-power Controller
DFR0355 SIM808 with Leonardo mainboard
DFR0392 DFRduino M0 Mainboard Arduino Compatible
DFR0398 Romeo BLE Quad Robot Controller
DFR0416 Bluno M0 Mainboard
DFR0575 Beetle ESP32
DFR0133 X-Board
DFR0162 X-Board V2
DFR0428 3.5 inches TFT Touchscreen for Raspberry Pi
DFR0494 Raspberry Pi UPS HAT
DFR0514 DFR0603 IIC 16X2 RGB LCD KeyPad HAT V1.0
DFR0524 5.5 HDMI OLED-Display with Capacitive Touchscreen V2.0
DFR0550 5'' TFT-Display with Touchscreen V1.0
DFR0591 raspberry pi e-ink display module V1.0
DFR0592 DC Motor Driver HAT
DFR0604 I O Expansion HAT for Pi zero V1.0
DFR0566 IO Expansion HAT for Raspberry Pi
DFR0528 UPS HAT for Raspberry Pi Zero
DFR0331 Romeo for Edison Controller
DFR0453 DFRobot CurieNano - A mini Genuino Arduino 101 Board
TEL0110 CurieCore intel® Curie Neuron Module
DFR0478 FireBeetle ESP32 IOT Microcontroller(V3.0) Supports Wi-Fi & Bluetooth
DFR0483 FireBeetle Covers-Gravity I O Expansion Shield
FireBeetle Covers-24×8 LED Matrix
TEL0121 FireBeetle Covers-LoRa Radio 433MHz
TEL0122 FireBeetle Covers-LoRa Radio 915MHz
TEL0125 FireBeetle Covers LoRa Radio 868MHz
DFR0489 FireBeetle ESP8266 IOT Microcontroller
DFR0492 FireBeetle Board-328P with BLE4.1
DFR0498 FireBeetle Covers-Camera&Audio Media Board
DFR0507 FireBeetle Covers-OLED12864 Display
DFR0508 FireBeetle Covers-DC Motor & Stepper Driver
DFR0511 FireBeetle Covers-ePaper Black&White Display Module
DFR0531 FireBeetle Covers-ePaper Black&White&Red Display Module
DFR0536 Micro bit Gamepad Expansion Board
DFR0548 Micro bit Driver Expansion Board
ROB0148 micro: Maqueen for micro:bit
ROB0150 Micro bit Circular RGB LED Expansion Board
MBT0005 micro IO-BOX
SEN0159 CO2 Sensor
DFR0049 DFRobot Gas Sensor
TOY0058 Barometric Pressure Sensor
SEN0220 Infrared CO2 Sensor 0-50000ppm
SEN0219 Gravity Infrared CO2 Sensor For Arduino
SEN0226 Gravity I2C BMP280 Barometer Sensor
SEN0231 Gravity HCHO Sensor
SEN0251 Gravity BMP280 Barometric Pressure Sensors
SEN0132 Carbon Monoxide Gas Sensor MQ7
SEN0032 Triple Axis Accelerometer Breakout - ADXL345
DFR0143 Triple Axis Accelerometer MMA7361
Triple Axis Accelerometer FXLN83XX Series
SEN0072 CMPS09 - Tilt Compensated Magnetic Compass
SEN0073 9 Degrees of Freedom - Razor IMU
DFR0188 Flymaple V1.1
SEN0224 Gravity I2C Triple Axis Accelerometer - LIS2DH
SEN0140 10 DOF Mems IMU Sensor V2.0
SEN0250 Gravity BMI160 6-Axis Inertial Motion Sensor
SEN0253 Gravity BNO055 + BMP280 intelligent 10DOF AHRS
SEN0001 URM37 V5.0 Ultrasonic Sensor
SEN0002 URM04 V2.0
SEN0004 SRF01 Ultrasonic sensor
SEN0005 SRF02 Ultrasonic sensor
SEN0006 SRF05 Ultrasonic sensor
SEN0007 SRF08 Ultrasonic Sensor
SEN0008 SRF10 Ultrasonic sensor
SEN0149 URM06-RS485 Ultrasonic
SEN0150 URM06-UART Ultrasonic
SEN0151 URM06-PULSE Ultrasonic
SEN0152 URM06-ANALOG Ultrasonic
SEN0153 URM07-UART Ultrasonic Sensor
SEN0246 URM08-RS485 Waterproof Sonar Range Finder
SEN0304 URM09 Ultrasonic Sensor (Gravity-I2C) (V1.0)
SEN0304 URM09 Ultrasonic Sensor (Gravity-I2C) (V1.0)
SEN0300 Water-proof Ultrasonic Sensor ULS
SEN0301 Water-proof Ultrasonic Sensor ULA
SEN0307 URM09 Ultrasonic Sensor Gravity Analog
SEN0311 A02YYUW Waterproof Ultrasonic Sensor
SEN0312 ME007YS Waterproof Ultrasonic Sensor
SEN0313 A01NYUB Waterproof Ultrasonic Sensor
DFR0066 SHT1x Humidity and Temperature Sensor
DFR0067 DHT11 Temperature and Humidity Sensor
SEN0137 DHT22 Temperature and humidity module
DFR0023 DFRobot LM35 Linear Temperature Sensor
DFR0024 Gravity DS18B20 Temperature Sensor Arduino Compatible V2
DFR0024 Gravity DS18B20 Temperature Sensor Arduino Compatible V2
SEN0114 Moisture Sensor
TOY0045 TMP100 Temperature Sensor
TOY0054 SI7021 Temperature and humidity sensor
SEN0206 IR Thermometer Sensor MLX90614
SEN0227 SHT20 I2C Temperature & Humidity Sensor Waterproof Probe
SEN0236 Gravity I2C BME280 Environmental Sensor Temperature, Humidity, Barometer
SEN0248 Gravity I2C BME680 Environmental Sensor VOC, Temperature, Humidity, Barometer
DFR0558 Gravity Digital High Temperature Sensor K-type
SEN0308 Waterproof Capacitive Soil Moisture Sensor
SEN0019 Adjustable Infrared Sensor Switch
SEN0042 DFRobot Infrared sensor breakout
SEN0143 SHARP GP2Y0A41SK0F IR ranger sensor 4-30cm
SEN0013 Sharp GP2Y0A02YK IR ranger sensor 150cm
SEN0014 Sharp GP2Y0A21 Distance Sensor 10-80cm
SEN0085 Sharp GP2Y0A710K Distance Sensor 100-550cm
DFR0094 Digital IR Receiver Module
DFR0095 DIGITAL IR Transmitter Module
SEN0018 Digital Infrared motion sensor
DFR0107 IR Kit
SEN0264 TS01 IR Thermal Sensor (4-20mA)
SEN0169 Analog pH Meter Pro
DFR0300-H Gravity: Analog Electrical Conductivity Sensor(K=10)
DFR0300 Gravity Analog Electrical Conductivity Sensor Meter V2 K=1
SEN0165 Gravity Analog ORP Meter
SEN0161-V2 Gravity Analog pH Sensor Meter Kit V2
SEN0161 PH meter
SEN0237 Gravity Analog Dissolved Oxygen Sensor
SEN0204 Non-contact Liquid Level Sensor XKC-Y25-T12V
SEN0205 Liquid Level Sensor FS-IR02
SEN0244 Gravity Analog TDS Sensor Meter For Arduino
SEN0249 Gravity Analog Spear Tip pH Sensor Meter Kit For Soil And Food Applications
SEN0121 Steam Sensor
SEN0097 Light Sensor
SEN0026 DFRobot Ambient Light Sensor
TOY0044 UV Sensor
SEN0172 LX1972 ambient light sensor
SEN0043 TEMT6000 ambient light sensor
SEN0175 UV Sensor v1.0-ML8511
SEN0228 Gravity I2C VEML7700 Ambient Light Sensor
SEN0022 DFRobot Grayscale Sensor
SEN0017 Line Tracking Sensor for Arduino V4
SEN0147 Smart Grayscale sensor
SEN0101 TCS3200 Color Sensor
SEN0212 TCS34725 I2C Color Sensor For Arduino

Introduction
Specification
Board Overview
Tutorial for Arduino
Read Data via I2C
Read Data via UART
FAQ
More Documents

SKU:TEL0157

Introduction



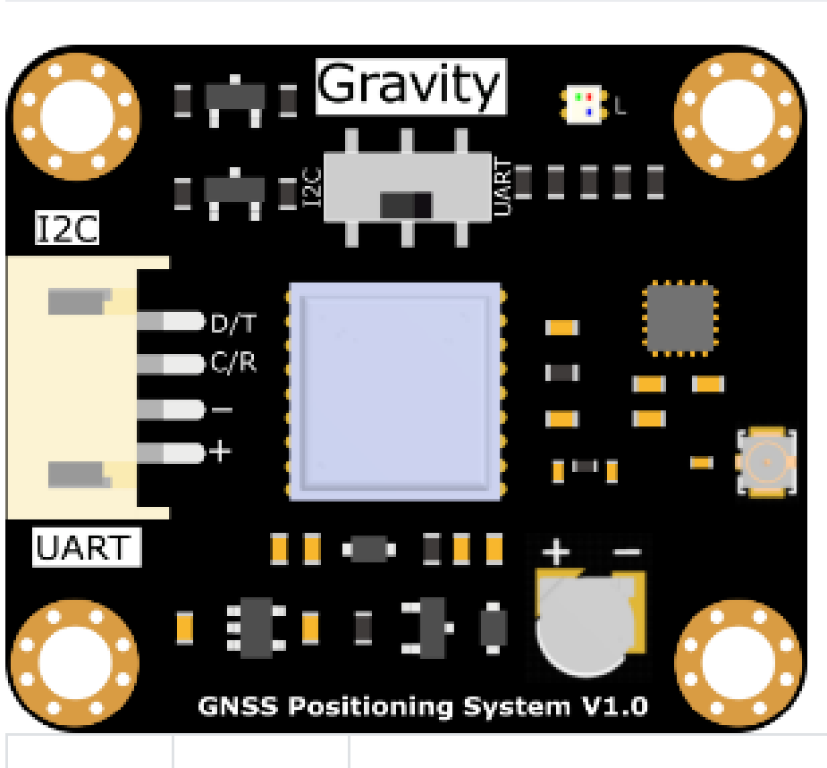
Global Navigation Satellite Systems (GNSS) provide critical timing and positioning functions for device operations. This Gravity: GNSS positioning module from DFRobot supports both single and multiple systems positioning. It is capable of quick delivery of position data like longitude, latitude, altitude and time. Compared with traditional single GPS positioning, the multi-system combination embraces higher precision and faster speed thanks to the increased number of visible and available satellites, which ensures stable and accurate performance even in complex urban environments.

With I2C and UART data outputs, the GNSS positioning module works well with main-controllers like Arduino, ESP32, and Raspberry Pi. It is applicable to outdoor positioning scenarios such as vehicle navigation, handheld positioning tracker, item tracking and weather station.

Specification

- Operating Voltage: 3.3 to 5.5V DC
- Output Signal: I2C/UART
- Positioning Accuracy: 2.0m CEP
- Time for First Positioning: 30s for cold start; 2s for hot start
- Power Consumption: 40mA
- Antenna Interface: IPEX1
- Antenna Frequency: 1561-1575.42MHz±3MHz
- Dimension: 27mm×37mm/1.06×1.46"

Board Overview



Num	Label	Description
1	D/T	I2C data line SDA/UART Data Transmitting- TX
2	C/R	I2C clock line SCL/UART Data Receiving-RX
3	-	GND
4	+	VCC

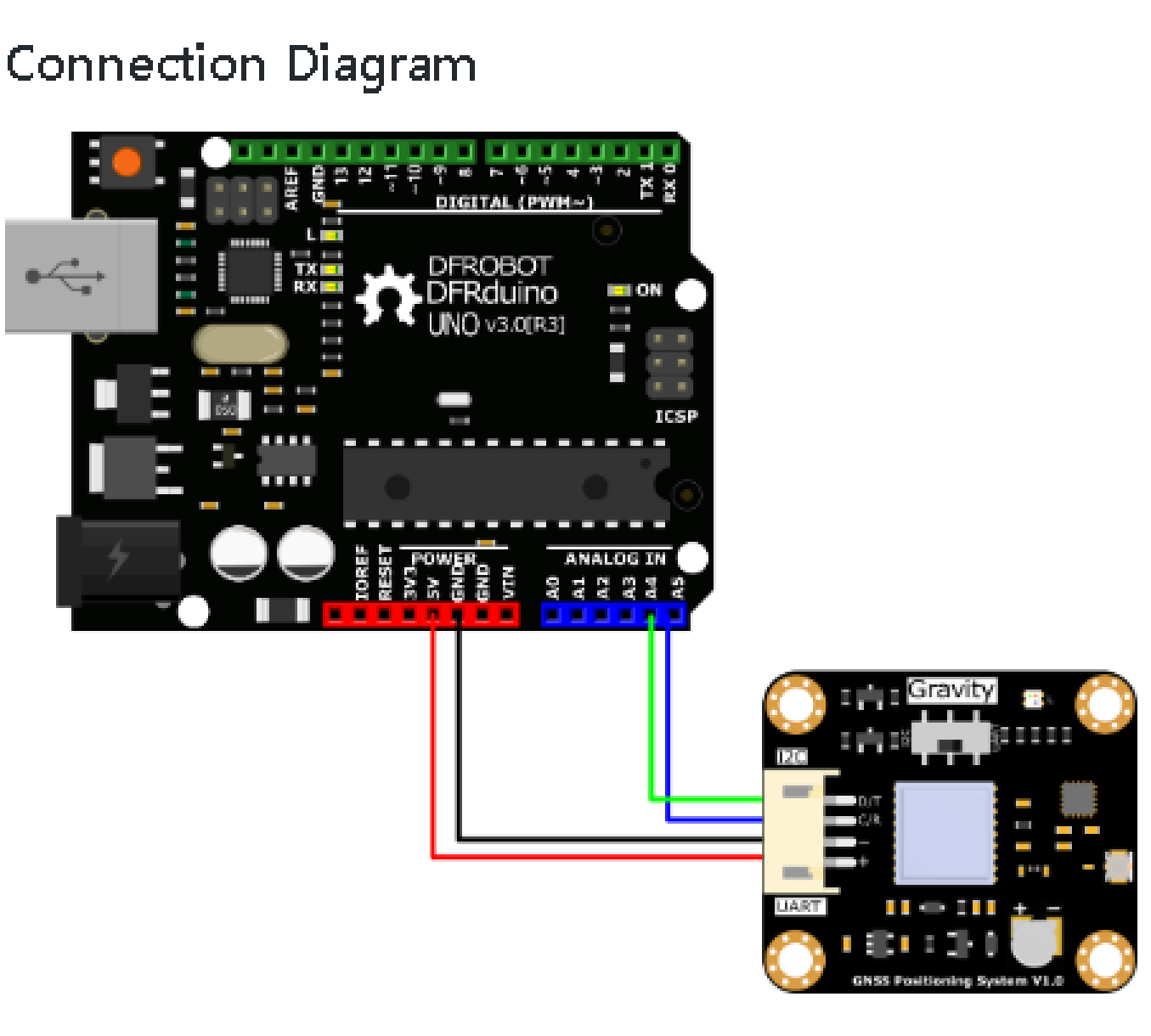
Tutorial for Arduino

Requirements

- **Hardware**
  - DFRduino UNO R3 (or similar) x 1
  - Gravity: GNSS Positioning Module x 1
- **Software**
  - Arduino IDE
  - Download and install the **DFRobot GNSS Library** (About how to install the library?)

Read Data via I2C

Connection Diagram



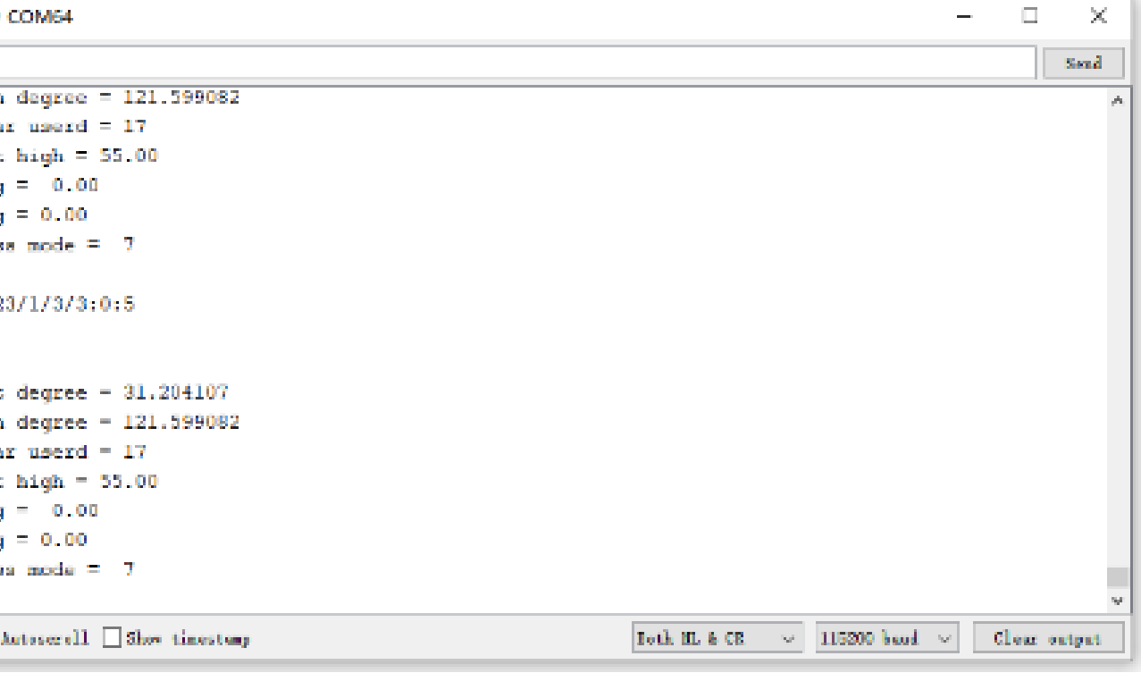
Sample Code

- Connect the module to Arduino according to the connection diagram above. It can also be used with Gravity I/O expansion board to prototype ideas faster.
- Change the select switch on the sensor to I2C.
- Download and install the DFRobot_GNSS library (About how to install the library?)
- Open Arduino IDE and upload the following code to Arduino UNO.
- Open the serial monitor of Arduino IDE, set the baud rate to 115200, and observe the printed result.

```
#include "DFRobot_GNSS.h"

DFRobot_GNSS_I2C gnss(&Wire ,GNSS_DEVICE_ADDR);
/*
#ifdef ESP_PLATFORM
  // ESP32 user hardware uart
  // RX io16
  // TX io17
  DFRobot_GNSS_UART gnss(&Serial2 ,9600);
#else
  // Arduino user software uart
  // RX io10
  // TX io11
  SoftwareSerial mySerial(10 ,11);
  DFRobot_GNSS_UART gnss(&mySerial ,9600);
#endif
*/
void setup()
{
  Serial.begin(115200);
  while(!gnss.begin()){
    Serial.println("NO Deivces !");
    delay(1000);
  }

  gnss.enablePower();

/** Set the galaxy to be used
 *   eGPS              USE gps
 *   eBeiDou           USE beidou
 *   eGPS_BeiDou       USE gps + beidou
 *   eGLONASS          USE glonass
 *   eGPS_GLONASS      USE gps + glonass
 *   eBeiDou_GLONASS   USE beidou +glonass
 *   eGPS_BeiDou_GLONASS USE gps + beidou + glonass
 */
  gnss.setGnss(eGPS_BeiDou_GLONASS);


  // gnss.setRgbOff();
  gnss.setRgbOn();
  // gnss.disablePower();
}

void loop()
{
  sTim_t utc = gnss.getUTC();
  sTim_t date = gnss.getDate();
  sLonLat_t lat = gnss.getLat();
  sLonLat_t lon = gnss.getLon();
  double high = gnss.getAlt();
  uint8_t starUserd = gnss.getNumSatUsed();
  double sog = gnss.getSog();
  double cog = gnss.getCog();

  Serial.println("");
  Serial.print(date.year);
  Serial.print("/");
  Serial.print(date.month);
  Serial.print("/");
  Serial.print(date.date);
  Serial.print("/");
  Serial.print(utc.hour);
  Serial.print(":");
  Serial.print(utc.minute);
  Serial.print(":");
  Serial.print(utc.second);
  Serial.println();
  Serial.println((char)lat.latDirection);
  Serial.println((char)lon.lonDirection);

  // Serial.print("lat DDMM.MMMMM = ");
  // Serial.println(lat.latitude, 5);
  // Serial.print(" lon DDDMM.MMMMM = ");
  // Serial.println(lon.lonitude, 5);
  Serial.print("lat degree = ");
  Serial.println(lat.latitudeDegree,6);
  Serial.print("lon degree = ");
  Serial.println(lon.lonitudeDegree,6);

  Serial.print("star userd = ");
  Serial.println(starUserd);
  Serial.print("alt high = ");
  Serial.println(high);
  Serial.print("sog =  ");
  Serial.println(sog);
  Serial.print("cog = ");
  Serial.println(cog);
  Serial.print("gnss mode =  ");
  Serial.println(gnss.getGnssMode());
  delay(1000);
}
```

Result

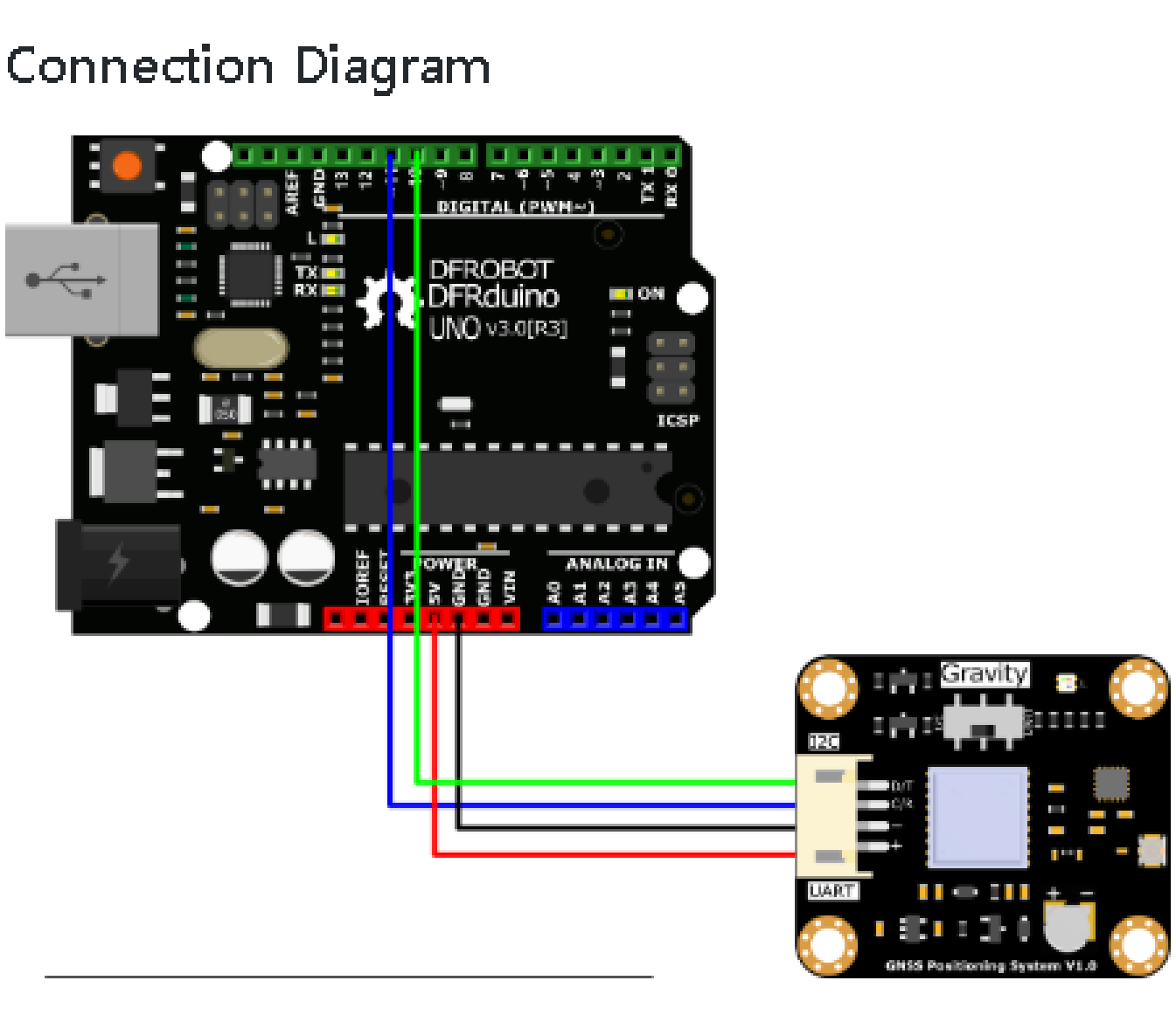
Open serial monitor to see the result.



```
lon degree = 121.599082
star userd = 17
alt high = 55.00
sog =   0.00
cog = 0.00
gnss mode =  7

2023/1/3/3:0:5
E
N
lat degree = 31.204107
lon degree = 121.599082
star userd = 17
alt high = 55.00
sog =   0.00
cog = 0.00
gnss mode =  7
```

Read Data via UART

Connection Diagram



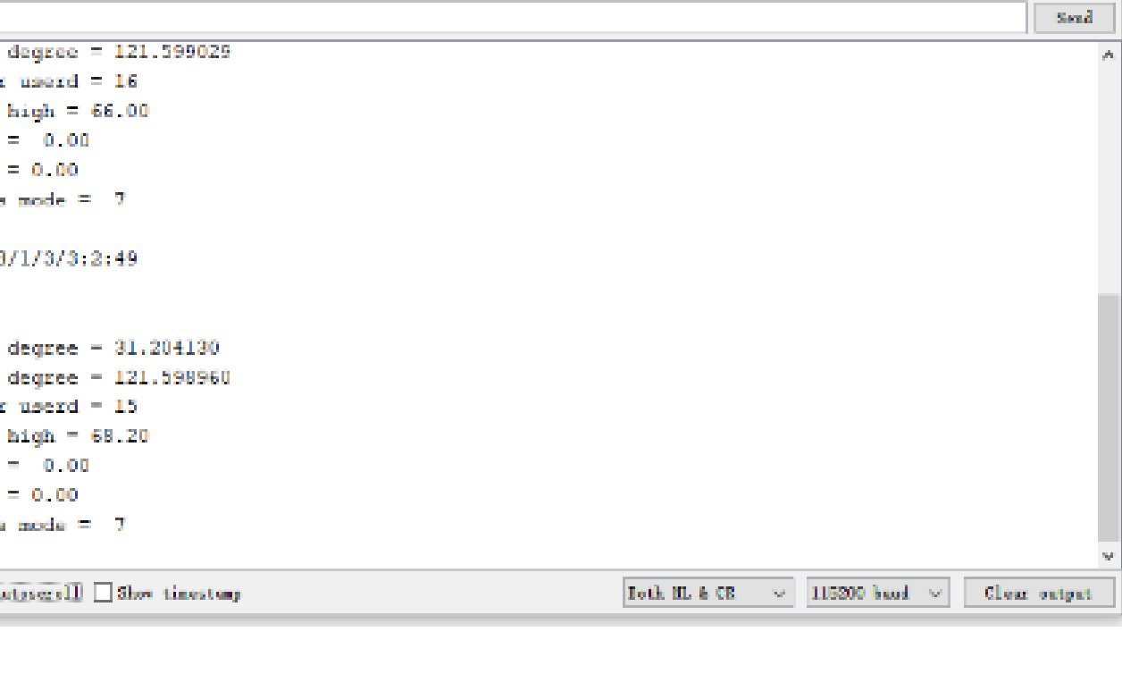
Sample Code

- Connect the module to Arduino according to the connection diagram above. It can also be used with Gravity I/O expansion board to prototype ideas faster.
- Change the select switch on the sensor to UART.
- Download and install the DFRobot_GNSS library (About how to install the library?)
- Open Arduino IDE and upload the following code to Arduino UNO.
- Open the serial monitor of Arduino IDE, set the baud rate to 115200, and observe the printed result.

```
 /*!
  * @file  getGNSS.ino
  * @copyright Copyright (c) 2010 DFRobot Co.Ltd (http
  * @license The MIT License (MIT)
  * @author ZhixinLiu(zhixin.liu@dfrobot.com)
  * @version V0.1
  * @date 2022-08-15
  * @url https://github.com/dfrobot/DFRobot_GNSS
  */

#include "DFRobot_GNSS.h"

//DFRobot_GNSS_I2C gnss(&Wire ,GNSS_DEVICE_ADDR);

#ifdef ESP_PLATFORM
  // ESP32 user hardware uart
  // RX io16
  // TX io17
  DFRobot_GNSS_UART gnss(&Serial2 ,9600);
#else
  // Arduino user software uart
  // RX io10
  // TX io11
  SoftwareSerial mySerial(10 ,11);
  DFRobot_GNSS_UART gnss(&mySerial ,9600);
#endif

void setup()
{
  Serial.begin(115200);
  while(!gnss.begin()){
    Serial.println("NO Deivces !");
    delay(1000);
  }

  gnss.enablePower();

/** Set the galaxy to be used
 *   eGPS              USE gps
 *   eBeiDou           USE beidou
 *   eGPS_BeiDou       USE gps + beidou
 *   eGLONASS          USE glonass
 *   eGPS_GLONASS      USE gps + glonass
 *   eBeiDou_GLONASS   USE beidou +glonass
 *   eGPS_BeiDou_GLONASS USE gps + beidou + glonass
 */
  gnss.setGnss(eGPS_BeiDou_GLONASS);


  // gnss.setRgbOff();
  gnss.setRgbOn();
  // gnss.disablePower();
}

void loop()
{
  sTim_t utc = gnss.getUTC();
  sTim_t date = gnss.getDate();
  sLonLat_t lat = gnss.getLat();
  sLonLat_t lon = gnss.getLon();
  double high = gnss.getAlt();
  uint8_t starUserd = gnss.getNumSatUsed();
  double sog = gnss.getSog();
  double cog = gnss.getCog();

  Serial.println("");
  Serial.print(date.year);
  Serial.print("/");
  Serial.print(date.month);
  Serial.print("/");
  Serial.print(date.date);
  Serial.print("/");
  Serial.print(utc.hour);
  Serial.print(":");
  Serial.print(utc.minute);
  Serial.print(":");
  Serial.print(utc.second);
  Serial.println();
  Serial.println((char)lat.latDirection);
  Serial.println((char)lon.lonDirection);

  // Serial.print("lat DDMM.MMMMM = ");
  // Serial.println(lat.latitude, 5);
  // Serial.print(" lon DDDMM.MMMMM = ");
  // Serial.println(lon.lonitude, 5);
  Serial.print("lat degree = ");
  Serial.println(lat.latitudeDegree,6);
  Serial.print("lon degree = ");
  Serial.println(lon.lonitudeDegree,6);

  Serial.print("star userd = ");
  Serial.println(starUserd);
  Serial.print("alt high = ");
  Serial.println(high);
  Serial.print("sog =  ");
  Serial.println(sog);
  Serial.print("cog = ");
  Serial.println(cog);
  Serial.print("gnss mode =  ");
  Serial.println(gnss.getGnssMode());
  delay(1000);
}
```

Result

Open serial monitor to see the result.



```
lon degree = 121.599025
star userd = 16
alt high = 66.00
sog =   0.00
cog = 0.00
gnss mode =  7

2023/1/3/3:2:49
E
N
lat degree = 31.204130
lon degree = 121.598960
star userd = 15
alt high = 68.20
sog =   0.00
cog = 0.00
gnss mode =  7
```

FAQ

For any questions, advice or cool ideas to share, please visit the **DFRobot Forum**.

More Documents

- Datasheet
- Schematics
- Dimensions & Component Layout

Get **Gravity: GNSS Positioning Module** from DFRobot Store or **DFRobot Distributor**.

Turn to the Top